

ROTEIRO DE LABORATÓRIO PRÁTICO

Revisão de Lógica Proposicional e Conectivos no GNU Octave

Disciplina: Matemática Discreta

Curso: Bacharelado em Sistemas de Informação

Professor: Eduardo Max Amaro Amaral

Instituição: IFES – Campus Colatina

1. Objetivos da Aula

Nesta aula prática, você revisará os conceitos fundamentais da lógica proposicional e implementará operações lógicas no GNU Octave. Os principais objetivos são:

- Revisar os conceitos de proposição e os conectivos lógicos: Negação ($\sim$), Conjunção ($\wedge$), Disjunção ($\vee$) e Disjunção Exclusiva ($\vee$ ou XOR).
- Mapear as operações lógicas formais para a sintaxe e funções nativas do GNU Octave ($\sim$, $\&$, $|$, xor).
- Implementar rotinas modulares em arquivos `.m` para geração e avaliação de tabelas-verdade.
- Desenvolver algoritmos para validação de expressões lógicas, equivalências e propriedades booleanas como atividades de entrega.

2. Revisão Teórica: Proposições e Conectivos Lógicos

Uma proposição é uma sentença declarativa que assume um valor lógico: **Verdadeiro (V/1)** ou **Falso (F/0)**, regida pelos princípios da **Não Contradição** (não pode ser V e F ao mesmo tempo) e do **Terceiro Excluído** (assume apenas V ou F, nunca um terceiro valor).

Conectivos Lógicos no GNU Octave

Conectivo / Operação	Símbolo Lógico	Operador no Octave	Definição Resumida
Negação (NOT)	$\sim p$	<code>~p</code>	Inverte o valor lógico de p.
Conjunção (AND)	$p \wedge q$	<code>p & q</code>	Verdadeira apenas se ambas as proposições forem verdadeiras.
Disjunção (OR)	$p \vee q$	<code>p q</code>	Falsa apenas se ambas as proposições forem falsas.
Disjunção Exclusiva (XOR)	$p \vee q$	<code>xor(p, q)</code>	Verdadeira se exatamente uma das proposições for verdadeira.

Tabela-Verdade Resumida

p	q	$\sim p$	$p \wedge q$	$p \vee q$	$p \vee q$ (XOR)
1	1	0	1	1	0
1	0	0	0	1	1
0	1	1	0	1	1
0	0	1	0	0	0

3. Parte 1: Prática Guiada no GNU Octave

No Octave, variáveis booleanas são tratadas como **1** (true) e **0** (false). As operações podem ser aplicadas diretamente em escalares ou em vetores.

Exemplo 1: Avaliação de Expressões no Prompt (Command Window)

```
>> p = 1; q = 0;  
>> ~p           % Negação: retorna 0  
>> p & q        % Conjunção: retorna 0  
>> p | q        % Disjunção: retorna 1  
>> xor(p, q)    % Disjunção Exclusiva: retorna 1
```

Exemplo 2: Função Geradora de Tabela-Verdade Básica

Crie o arquivo `tabela_conjuncao.m`:

```
function T = tabela_conjuncao()  
    % Retorna a tabela-verdade completa para p AND q  
    p = [1; 1; 0; 0];  
    q = [1; 0; 1; 0];  
    resultado = p & q;  
    T = [p, q, resultado];  
endfunction
```

4. Parte 2: Atividades Práticas Básicas (Laboratório)

Cada função deve ser implementada em seu respectivo arquivo `.m`.

Atividade 1: Tabela Completa dos Conectivos Fundamentais

- **Arquivo:** `tabela_conectivos.m`

- **Descrição:** Crie uma função sem parâmetros que gere e retorne uma matriz 4×6 contendo as seguintes colunas:

1. p
2. q
3. $\sim p$
4. $p \wedge q$
5. $p \vee q$
6. $p \vee q$

- **Assinatura:** `function T = tabela_conectivos()`

Atividade 2: Verificador de Disjunção Exclusiva Manual

- **Arquivo:** `meu_xor.m`

• **Descrição:** Implemente o conector XOR sem utilizar a função nativa `xor()`, usando apenas os conectivos fundamentais: $(p \vee q) \wedge \sim(p \wedge q)$.

- **Assinatura:** `function res = meu_xor(p, q)`

Atividade 3: Verificador das Leis de De Morgan

- **Arquivo:** `de_morgan_conjuncao.m`

• **Descrição:** Prove a equivalência $\sim(p \wedge q) \Leftrightarrow (\sim p \vee \sim q)$ avaliando todos os pares possíveis. A função deve retornar 1 (se a equivalência for válida para todas as linhas) e 0 caso contrário.

- **Assinatura:** `function valido = de_morgan_conjuncao()`

5. Parte 3: Atividades de Entrega (Tarefas Mais Complexas)

Atividade 4: Avaliador Geral de Fórmulas Proposicionais

- **Arquivo:** `avalia_expressao.m`

- **Descrição:**

Crie uma função que receba três vetores coluna de valores lógicos de mesmo tamanho representando as proposições p , q e r , e devolva a coluna resultante da fórmula proposicional: $E = (p \wedge \sim q) \vee (q \vee r)$

- **Assinatura:** `function E = avalia_expressao(p, q, r)`

- **Exemplo de Entrada:**

```
p = [1; 1; 1; 1; 0; 0; 0; 0];
q = [1; 1; 0; 0; 1; 1; 0; 0];
r = [1; 0; 1; 0; 1; 0; 1; 0];
```

Atividade 5: Classificador Semântico (Tautologia, Contradição ou Contingência)

- **Arquivo:** `classifica_proposicao.m`

- **Descrição:** Crie uma função que receba um vetor lógico `coluna_resultado` (a última coluna de uma tabela-verdade) e classifique a fórmula retornando uma string:

- "TAUTOLOGIA": se todas as saídas forem verdadeiras (1).
- "CONTRADICAO": se todas as saídas forem falsas (0).
- "CONTINGENCIA": se houver mescla de saídas verdadeiras e falsas.

- **Assinatura:**

```
function tipo = classifica_proposicao(coluna_resultado)
```

- **Dica:** Utilize as funções `all()` e `any()`.

Atividade 6: Testador de Equivalência Lógica por Tabela-Verdade

- **Arquivo:** `sao_equivalentes.m`

- **Descrição:** Dadas duas matrizes booleanas ou vetores coluna resultantes V_1 e V_2 representando duas proposições compostas sobre as mesmas premissas, verifique se elas são logicamente equivalentes ($V_1 \Leftrightarrow V_2$).

- A função deve retornar 1 se forem equivalentes e exibir na tela `disp("As proposicoes sao logicamente equivalentes")`.

- Deve retornar 0 se não forem equivalentes e exibir o número de casos em que os valores divergiram.

- **Assinatura:** `function eq = sao_equivalentes(V1, V2)`

Critérios de Avaliação das Atividades de Entrega

- **Modularização:** Um arquivo `.m` individual para cada função.

- **Correção Lógica:** Conformidade rigorosa com as tabelas-verdade da lógica proposicional.
- **Vetorização:** Aproveitamento das operações lógicas vetoriais do Octave.